

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

Proyecto

Organización Computacional

Documentación de la *Casa Inteligente con Control de Ambientes y Ventilador Automatizado*

Carnet	Integrante
202406918	Claudia Maribel Tigüilá Tecum
202403929	Emiliana Elizabeth Pú Lara
202300476	Alex Ricardo Castañeda Rodríguez
201904522	Byron Manuel Hernández López

Grupo 21

Abril de 2026

Contents

1	Resumen	3
2	Marco Formativo	3
2.1	Valor	3
2.2	Competencias	3
2.3	Objetivo SMART	3
3	Enunciado del Proyecto	4
3.1	Descripción del problema a resolver	4
3.2	Alcance del proyecto	4
4	Requisitos Funcionales	4
4.1	Control de escenas	4
4.2	Persistencia en EEPROM	5
4.3	Carga de configuración desde archivo .org	5
4.4	Control remoto por Bluetooth y serial	5
4.5	Retroalimentación local	6
4.6	Actuadores y salidas	6
5	Arquitectura de la Solución	6
5.1	Descripción general	6
5.2	Unidad central de procesamiento	7
5.3	Apps de apoyo	8
6	Formato del archivo .org	8
6.1	Sintaxis reconocida por el sketch	8
6.2	Valores válidos	9
6.3	Ejemplo completo	10
6.4	Errores detectados	10
7	Tabla de direcciones EEPROM	11
8	Implementación Física	12

8.1	Diagrama general y esquema	12
8.2	Unidad central y periféricos	13
8.3	Módulos de salida	15
8.4	Interfaces del sistema	17
9	Fragmentos clave del código Arduino	22
9.1	Aplicación de escenas	22
9.2	Escenas por defecto	23
9.3	Comandos operativos	23
10	Recursos y herramientas utilizadas	24
10.1	Hardware	24
10.2	Software	24
11	Cronograma de trabajo	25
12	Presupuesto	25
13	Resultados y observaciones	25
14	Conclusiones	26
15	Anexos	26
15.1	Archivos de apoyo	26
15.2	Pendientes para cierre final	26

1 Resumen

En este proyecto se desarrolló una maqueta de casa inteligente capaz de controlar cinco ambientes de iluminación, un ventilador DC y una puerta automatizada con servomotor. El sistema utiliza un Arduino como unidad central, memoria EEPROM para persistir escenas, una pantalla LCD I2C para retroalimentación local, una aplicación de escritorio para cargar configuraciones .org por USB y una aplicación móvil para activar modos por Bluetooth.

La solución cubre la lógica pedida en el PDF del proyecto: escenas persistentes, control remoto, validación de configuración, retroalimentación visual y documentación técnica del circuito, del flujo de datos y de los activos de software involucrados.

2 Marco Formativo

2.1 Valor

El proyecto integra conceptos de organización computacional con una implementación embebida real. La práctica de persistencia en EEPROM, comunicación serial, control de periféricos y organización modular del software permite aplicar teoría en un sistema funcional y verificable.

2.2 Competencias

Durante el desarrollo del proyecto se fortalecieron las siguientes competencias:

- Diseño e integración de sistemas embebidos con Arduino.
- Uso de memoria EEPROM para almacenamiento persistente.
- Definición y validación de formatos de entrada estructurados.
- Control de actuadores y salidas digitales desde una arquitectura centralizada.
- Desarrollo de interfaces de apoyo en escritorio y móvil.
- Documentación técnica de circuitos, flujo de datos y entregables.

2.3 Objetivo SMART

Diseñar, integrar y documentar antes del 2 de mayo de 2026 una maqueta funcional de casa inteligente con cinco ambientes, control de ventilador y puerta, escenas almacenadas en EEPROM, carga de configuración desde archivo .org y activación remota por Bluetooth, dejando evidencia del circuito, del código y del flujo completo de operación.

3 Enunciado del Proyecto

3.1 Descripción del problema a resolver

El proyecto plantea automatizar una vivienda a escala mediante escenas de iluminación predefinidas, persistencia de configuración y control remoto. El sistema debe permitir que un usuario defina modos desde una computadora por medio de un archivo `.org`, guarde esos modos en EEPROM, los active por Bluetooth o por serial y obtenga retroalimentación inmediata mediante LCD y LEDs de estado.

Además del control de luces, la solución incorpora un ventilador DC y una puerta automatizada con servomotor, con lo cual el sistema deja de ser solo un ejemplo de persistencia de datos y se convierte en una maqueta domótica funcional.

3.2 Alcance del proyecto

El repositorio cubre cuatro bloques principales:

- El sketch de Arduino que implementa persistencia, lectura de comandos, control de LEDs, ventilador, LCD y puerta.
- La aplicación de escritorio `EEPROM_Liquid_Controller`, usada para seleccionar y enviar archivos `.org` por USB.
- La aplicación móvil `eeeprom_liquid_remote`, usada para activar modos desde Bluetooth clásico.
- Los diagramas y evidencias visuales del montaje y de las interfaces del sistema.

4 Requisitos Funcionales

Con base en el PDF del proyecto, los requisitos funcionales principales atendidos por la solución son los siguientes:

4.1 Control de escenas

- `modo_fiesta`
- `modo_relajado`
- `modo_noche`
- `encender_todo`
- `apagar_todo`

Cada escena almacena estado del ventilador, patrón de LEDs y mensaje para LCD.

4.2 Persistencia en EEPROM

Cada escena se guarda en una dirección fija de EEPROM mediante una estructura con los siguientes campos:

- `magic`
- `fan`
- `ledPattern`
- `message[33]`

El sistema además precarga modos por defecto si no detecta una configuración válida.

4.3 Carga de configuración desde archivo `.org`

El sketch reconoce el flujo:

1. `conf_ini`
2. Definición de uno o varios modos
3. `conf:fin`

Durante la carga:

- Se ignoran líneas vacías y comentarios con `//`.
- Se valida que cada modo tenga ventilador y patrón de LEDs.
- Si hay error de sintaxis, se cancela la carga y se reporta `UPLOAD_ERROR`.
- Si la carga finaliza correctamente, el LED verde parpadea tres veces y el LCD muestra confirmación.

4.4 Control remoto por Bluetooth y serial

Comandos soportados por el sketch:

- `modo_fiesta`
- `modo_relajado`
- `modo_noche`
- `encender_todo`
- `apagar_todo`

- estado
- abrir_puerta
- cerrar_puerta

4.5 Retroalimentación local

- LCD I2C 16x2 con mensaje del modo y estado del ventilador.
- LED azul para sistema listo.
- LED verde para carga exitosa.
- LED rojo para errores.

4.6 Actuadores y salidas

- Cinco salidas de iluminación: sala, comedor, cocina, baño y habitación.
- Un motor DC para ventilador.
- Un servomotor para apertura y cierre de puerta.
- Un botón físico con *debounce* para alternar la puerta.

5 Arquitectura de la Solución

5.1 Descripción general

La arquitectura se divide en tres capas operativas:

1. Unidad central Arduino: recibe comandos por serial, administra EEPROM y controla salidas físicas.
2. Interfaz de escritorio: prepara y transmite archivos `.org` al Arduino por USB.
3. Interfaz móvil: envía comandos por Bluetooth clásico para activar modos o consultar estado.

Table 1: Flujo general del sistema.

Origen	Destino y función
Archivo <code>.org</code>	App de escritorio → Serial USB → Arduino → EEPROM
Teléfono Android	Bluetooth HC-06 → Arduino → luces, LCD, ventilador y puerta
EEPROM	Arduino → recuperación y aplicación del modo activo

5.2 Unidad central de procesamiento

El archivo principal del sistema es `arduino/eprom_liquid_controller/eprom_liquid_controller.ino`. En él se definen los pines, el protocolo de carga del archivo `.org`, el mapa de EEPROM, la gestión de escenas y el control de actuadores.

Table 2: Distribución de pines según el sketch.

Pin	Uso
0	<code>rx_bluetooth</code>
1	<code>tx_bluetooth</code>
2	<code>led_azul</code>
3	<code>led_verde</code>
4	<code>led_rojo</code>
5	<code>luces_sala</code>
6	<code>luces_comedor</code>
7	<code>luces_cocina</code>
8	<code>luces_bano</code>
9	<code>servo_puerta</code>
10	<code>motor_ventilador</code>
11	<code>boton_puerta</code>
12	<code>luces_habitacion</code>
A4	SDA del LCD I2C
A5	SCL del LCD I2C

La tabla anterior corresponde de forma directa con las constantes definidas en el sketch. Esto permite relacionar el diagrama en Proteus con la implementación real y comprobar que cada bloque mostrado en la documentación sí tiene una salida física asociada en el Arduino.

```
const int led_azul = 2;
const int led_verde = 3;
const int led_rojo = 4;
const int luces_sala = 5;
const int luces_comedor = 6;
const int luces_cocina = 7;
const int luces_bano = 8;
const int servo_puerta = 9;
const int motor_ventilador = 10;
const int boton_puerta = 11;
const int luces_habitacion = 12;
```

Figure 1: Definición de pines usada por el sketch Arduino.

5.3 Apps de apoyo

Table 3: Bloques de software del repositorio.

Componente	Tecnología	Función
EEPROM_Liquid_Controller	Python + Tkinter	Selección de puerto, carga de archivo .org y envío serial
eeeprom_liquid_remote	Flutter	Conexión Bluetooth y envío de comandos de modo
Sketch Arduino	C++ para Arduino	Procesamiento central, EEPROM, LCD, luces, ventilador y puerta

Las aplicaciones de apoyo sirven para cargar configuraciones y enviar comandos. La lógica del sistema sigue concentrada en el Arduino: allí se validan escenas, se escriben datos en EEPROM y se activan luces, ventilador y puerta.

6 Formato del archivo .org

6.1 Sintaxis reconocida por el sketch

Las reglas observadas en el sketch son las siguientes:

- El archivo debe iniciar con `conf_ini`.
- Debe cerrar con `conf:fin`.
- Los comentarios usan `//`.
- Cada modo se define por su nombre.
- Para cada modo deben declararse `Ventilador:` y `LED'S:`.
- `Mensaje en LCD:` es opcional; si no aparece, el sistema usa un mensaje por defecto.

La validación se hace durante la lectura serial, línea por línea. Así, cuando aparece un error, el sistema puede marcarlo enseguida en el LCD, en el LED rojo y en la salida serial.

```

void processLine(String rawLine) {
    String line = stripComment(rawLine);
    line.trim();
    if (line.length() == 0) return;

    if (line == "conf_ini") {
        beginUpload();
        return;
    }

    if (uploadActive) {
        processUploadLine(line);
        return;
    }

    processCommand(line);
}

```

Figure 2: Separación entre modo de carga y modo de comandos en el sketch.

6.2 Valores válidos

Table 4: Valores reconocidos por el formato .org.

Campo	Valores válidos
Modo	modo_fiesta, modo_relajado, modo_noche, encender_todo, apagar_todo
Ventilador	ON, OFF
LEDs	ON, OFF, Alternandose

6.3 Ejemplo completo

```
// Configuracion de modos para el sistema de control
conf_ini

modo_fiesta
Mensaje en LCD: "Modo:  FIESTA."
Ventilador:  ON
LED'S: Alternandose

modo_relajado
Mensaje en LCD: "Modo:  RELAJADO."
Ventilador:  OFF
LED'S: ON

modo_noche
Mensaje en LCD: "Modo:  NOCHE."
Ventilador:  OFF
LED'S: OFF

encender_todo
Mensaje en LCD: "Modo:  TODO ON."
Ventilador:  ON
LED'S: ON

apagar_todo
Mensaje en LCD: "Modo:  TODO OFF."
Ventilador:  OFF
LED'S: OFF

conf:fin
```

Figure 3: Ejemplo de archivo .org compatible con el sistema.

6.4 Errores detectados

El sketch cancela la carga si detecta:

- modo no definido
- modo incompleto
- línea inválida
- ventilador inválido
- LEDs inválidos
- archivo vacío

Si aparece uno de estos fallos, la carga se detiene y el sistema muestra el motivo tanto en pantalla como por serial.

```
void failUpload(const char *reason) {
    uploadActive = false;
    currentUploadMode = NO_MODE;
    systemError = true;
    digitalWrite(led_azul, LOW);
    digitalWrite(led_verde, LOW);
    digitalWrite(led_rojo, HIGH);
    showLcd("Error archivo", reason);
    Serial.print("UPLOAD_ERROR:");
    Serial.println(reason);
}
```

Figure 4: Rutina de error durante la carga del archivo .org.

7 Tabla de direcciones EEPROM

La EEPROM quedó definida con direcciones fijas por modo. Cada escena inicia en una posición constante y desde allí se escribe la estructura completa de 36 bytes: 1 byte para `magic`, 1 byte para `fan`, 1 byte para `ledPattern` y 33 bytes para `message`.

Table 5: Direcciones fijas de almacenamiento de escenas en EEPROM.

Modo	Dirección fija	Valores por defecto
<code>modo_fiesta</code>	0	Ventilador ON, LEDs Alternandose, mensaje Modo: FIESTA.
<code>modo_relajado</code>	36	Ventilador OFF, LEDs ON, mensaje Modo: RELAJADO.
<code>modo_noche</code>	72	Ventilador OFF, LEDs OFF, mensaje Modo: NOCHE.
<code>encender_todo</code>	108	Ventilador ON, LEDs ON, mensaje Modo: TODO ON.
<code>apagar_todo</code>	144	Ventilador OFF, LEDs OFF, mensaje Modo: TODO OFF.

Estas direcciones están definidas de forma literal en el sketch. Por eso aquí se toman como posiciones fijas:

```
const int EEPROM_ADDR_FIESTA = 0;
const int EEPROM_ADDR_RELAJADO = 36;
const int EEPROM_ADDR_NOCHE = 72;
const int EEPROM_ADDR_ENCENDER_TODO = 108;
const int EEPROM_ADDR_APAGAR_TODO = 144;
```

Figure 5: Definición explícita de direcciones fijas en EEPROM.

La escritura persistente también se hace sobre esas posiciones mediante `EEPROM.put()`, usando la estructura `SceneConfig`. Así, cada modo conserva su configuración incluso después de reiniciar el Arduino.

```
void writeSceneToEeprom(byte mode, PendingScene scene) {
    SceneConfig stored;
    stored.magic = EEPROM_MAGIC;
    stored.fan = scene.fan;
    stored.ledPattern = scene.ledPattern;
    copyMessage(String(scene.message), stored.message);
    EEPROM.put(eepromAddress(mode), stored);
}
```

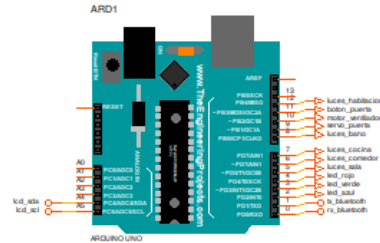
Figure 6: Rutina de escritura de escenas en EEPROM.

8 Implementación Física

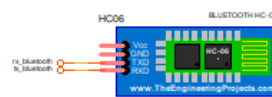
8.1 Diagrama general y esquema

La vista general ordena el proyecto por módulos. Arriba están los bloques de control y monitoreo; abajo, las salidas físicas de la casa. Sirve para ver de un golpe cómo se conectan Arduino, Bluetooth, LCD, iluminación, ventilación y puerta.

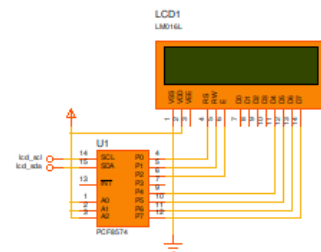
Unidad Central de Procesamiento



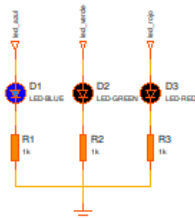
Receptor Bluetooth



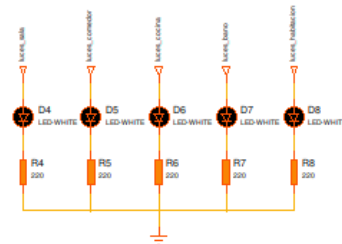
Interfaz de Monitoreo



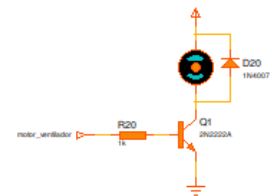
Indicadores de Configuración



Iluminación Domótica por Zonas



Control de Ventilación



Sistema de Control de Acceso

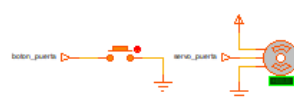


Figure 7: Vista modular general del sistema.

8.2 Unidad central y periféricos

La unidad central mostrada en Proteus corresponde al bloque que arranca el sistema. Desde aquí se configuran pines, se habilita la comunicación serial, se conecta el servomotor, se inicializa el LCD y se cargan escenas por defecto si la EEPROM aún no tiene datos válidos.

Unidad Central de Procesamiento

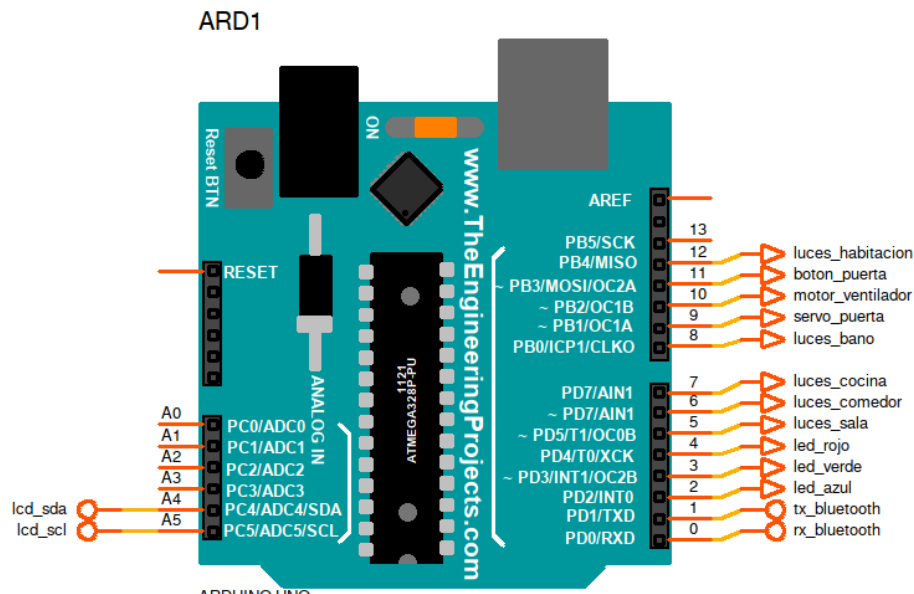


Figure 8: Unidad central de procesamiento basada en Arduino.

```
void setup() {  
    Serial.begin(BAUDRATE);  
    doorServo.attach(servo_puerta);  
    closeDoor();  
    lcd.init();  
    lcd.backlight();  
    setReadyState();  
    seedDefaultScenesIfNeeded();  
    applyMode(MODE_APAGAR_TODO);  
}
```

Figure 9: Secuencia principal de inicialización del sistema.

El receptor HC-06 es el canal de control remoto. Por aquí llegan textos como `modo_fiesta`, `modo_noche` o `estado`. El sketch los procesa igual que si vinieran del monitor serial.

Receptor Bluetooth

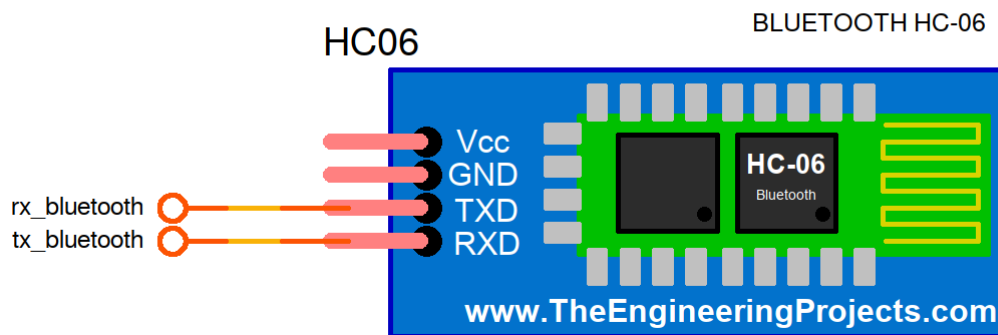


Figure 10: Receptor Bluetooth HC-06.

La pantalla LCD se usa para mostrar el estado del sistema. El Arduino la actualiza cada vez que cambia de modo o detecta un problema de configuración.

Interfaz de Monitoreo

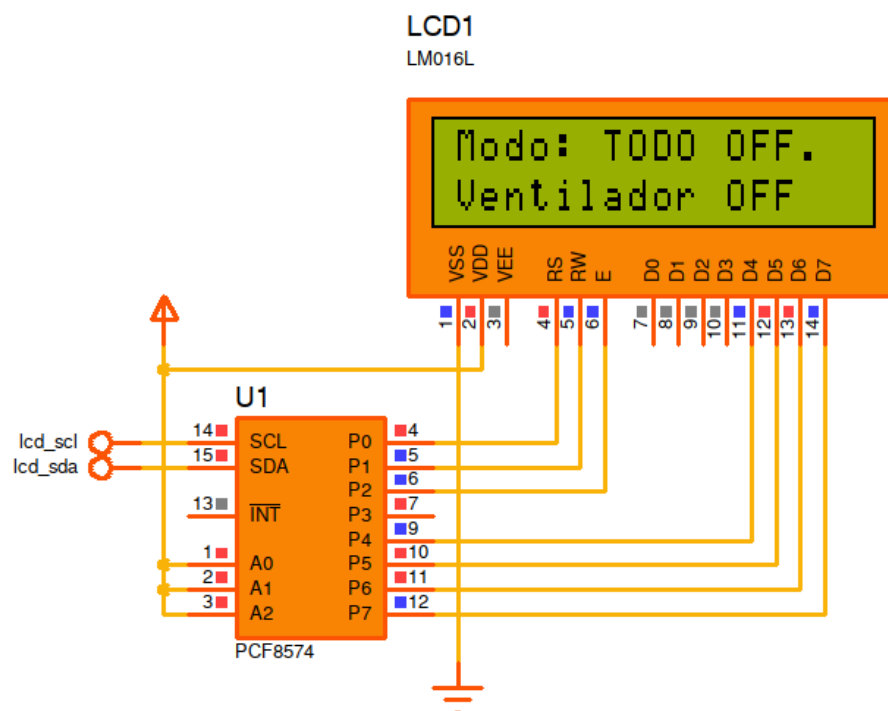


Figure 11: Interfaz de monitoreo con LCD I2C.

```
showLcd(scene.message,
        scene.fan == FAN_ON ? "Ventilador ON"
                           : "Ventilador OFF");
```

Figure 12: Actualización del LCD al aplicar una escena.

8.3 Módulos de salida

El bloque de indicadores resume tres estados del sistema: azul para listo, verde para carga exitosa y rojo para error. Estas salidas coinciden con las rutinas `setReadyState()`, `blinkGreenSuccess()` y `failUpload()` del sketch.

Indicadores de Configuración

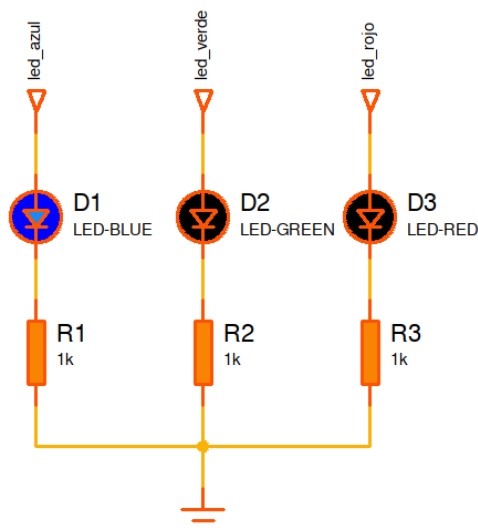


Figure 13: Indicadores de configuración.

La iluminación por zonas representa cinco salidas independientes: sala, comedor, cocina, baño y habitación. El proyecto no trabaja con un único LED general, sino con un conjunto de ambientes que pueden quedar todos encendidos, todos apagados o alternándose en el modo fiesta.

Iluminación Domótica por Zonas

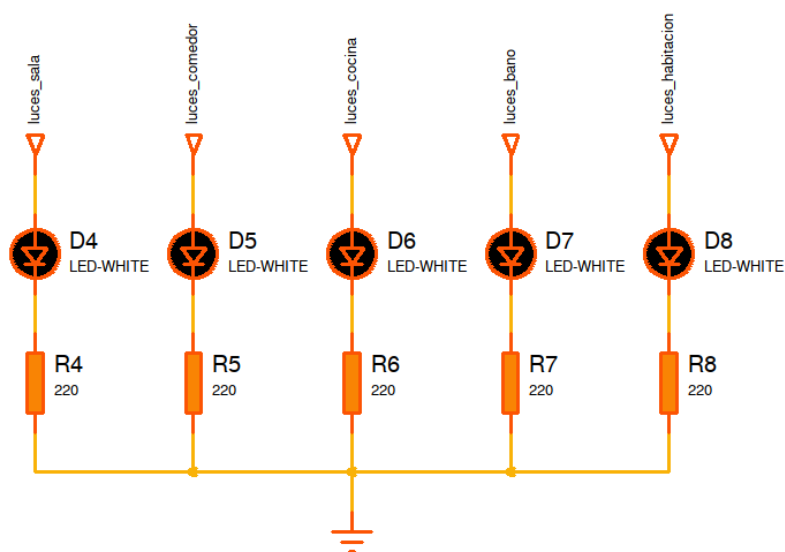


Figure 14: Iluminación domótica por zonas.


```

void setAllRoomLights(byte state) {
    digitalWrite(luces_sala, state);
    digitalWrite(luces_comedor, state);
    digitalWrite(luces_cocina, state);
    digitalWrite(luces_bano, state);
    digitalWrite(luces_habitacion, state);
}

```

Figure 15: Rutina que controla simultáneamente los cinco ambientes.

En el modo fiesta, el sketch no deja todas las luces fijas; las alterna usando `millis()`. Por eso este bloque no se queda en una conexión de LEDs, sino que responde a un patrón temporal real.

```

if (now - lastAlternateTick < 450) return;
lastAlternateTick = now;
alternatePhase = !alternatePhase;
digitalWrite(luces_sala, alternatePhase ? HIGH : LOW);
digitalWrite(luces_comedor, alternatePhase ? LOW : HIGH);
digitalWrite(luces_cocina, alternatePhase ? HIGH : LOW);
digitalWrite(luces_bano, alternatePhase ? LOW : HIGH);
digitalWrite(luces_habitacion, alternatePhase ? HIGH : LOW);

```

Figure 16: Patrón alternado de LEDs usado para modo_fiesta.

El módulo de ventilación se controla como una salida digital simple. El Arduino solo decide si el ventilador está encendido o apagado según lo guardado en la escena activa.

Control de Ventilación

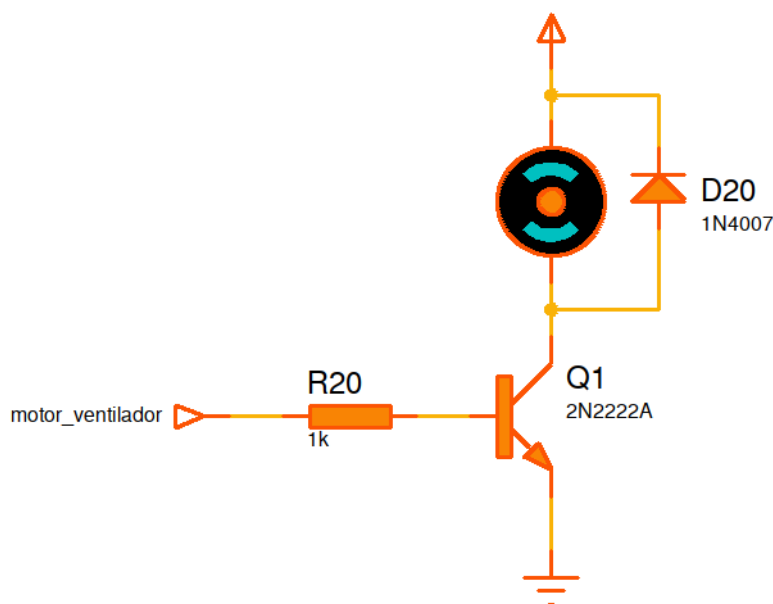


Figure 17: Control de ventilación con motor DC.

```
digitalWrite(motor_ventilador ,
            scene.fan == FAN_ON ? HIGH : LOW);
```

Figure 18: Activación directa del ventilador desde la escena actual.

El sistema de control de acceso muestra la parte mecánica de la puerta. En el sketch, el botón físico conectado al pin 11 pasa por una lógica de antirrebote y luego alterna entre abrir y cerrar.

Sistema de Control de Acceso

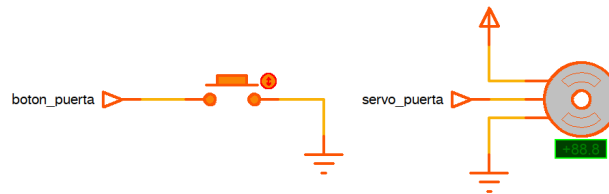


Figure 19: Sistema de control de acceso para la puerta.

```
if ((millis() - lastDoorDebounce) > 40) {
  if (reading != stableDoorButtonState) {
    stableDoorButtonState = reading;
    if (stableDoorButtonState == LOW) {
      if (doorOpen) closeDoor();
      else openDoor();
    }
  }
}
```

Figure 20: Lógica de antirrebote y alternancia usada para la puerta.

8.4 Interfaces del sistema

La pantalla inicial de la aplicación de escritorio es el punto de entrada para cargar escenas persistentes. Desde aquí el usuario selecciona puerto, prepara el archivo y lo envía al Arduino.

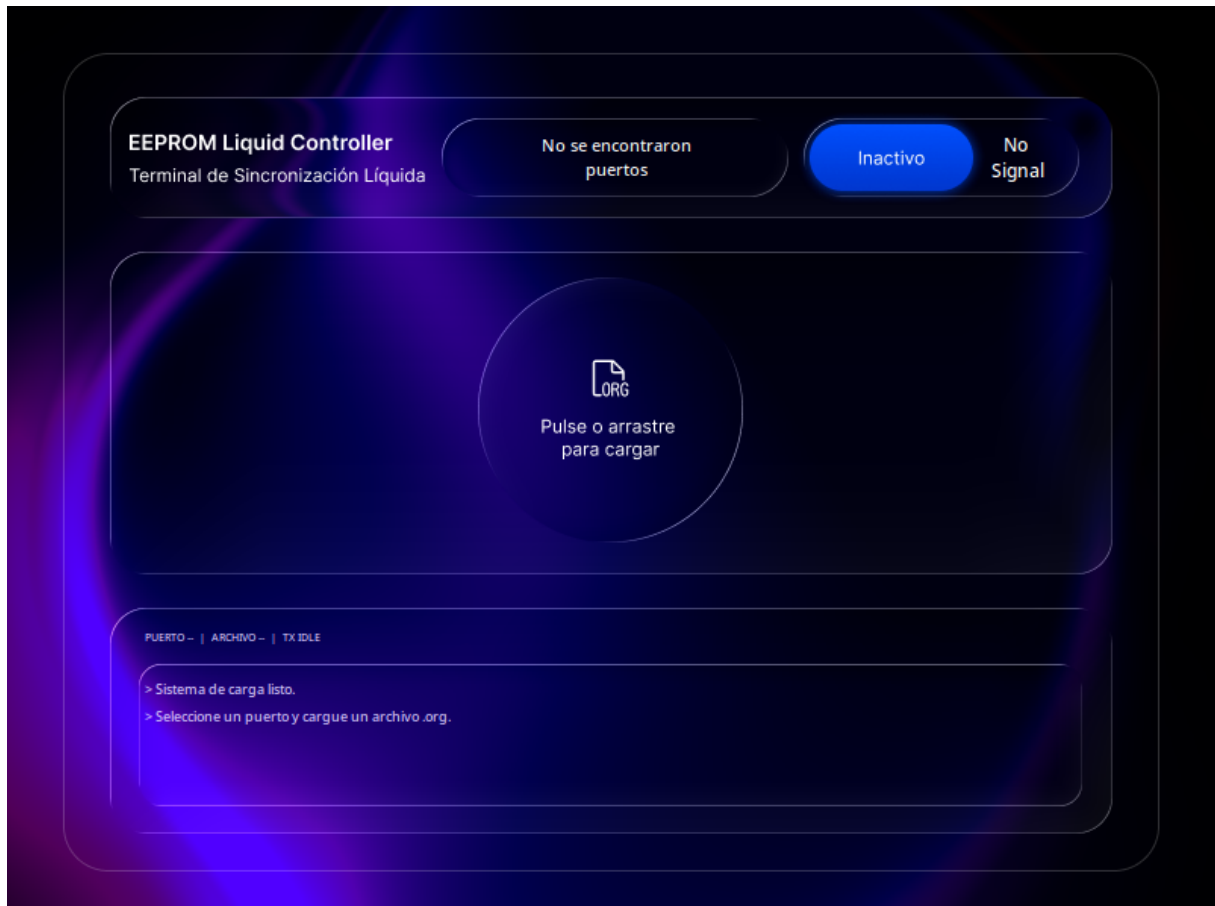


Figure 21: Pantalla inicial de la aplicación de escritorio.

La segunda captura muestra el archivo ya cargado en la interfaz. El contenido `.org` se trabaja como texto editable y visible antes de llegar al Arduino.

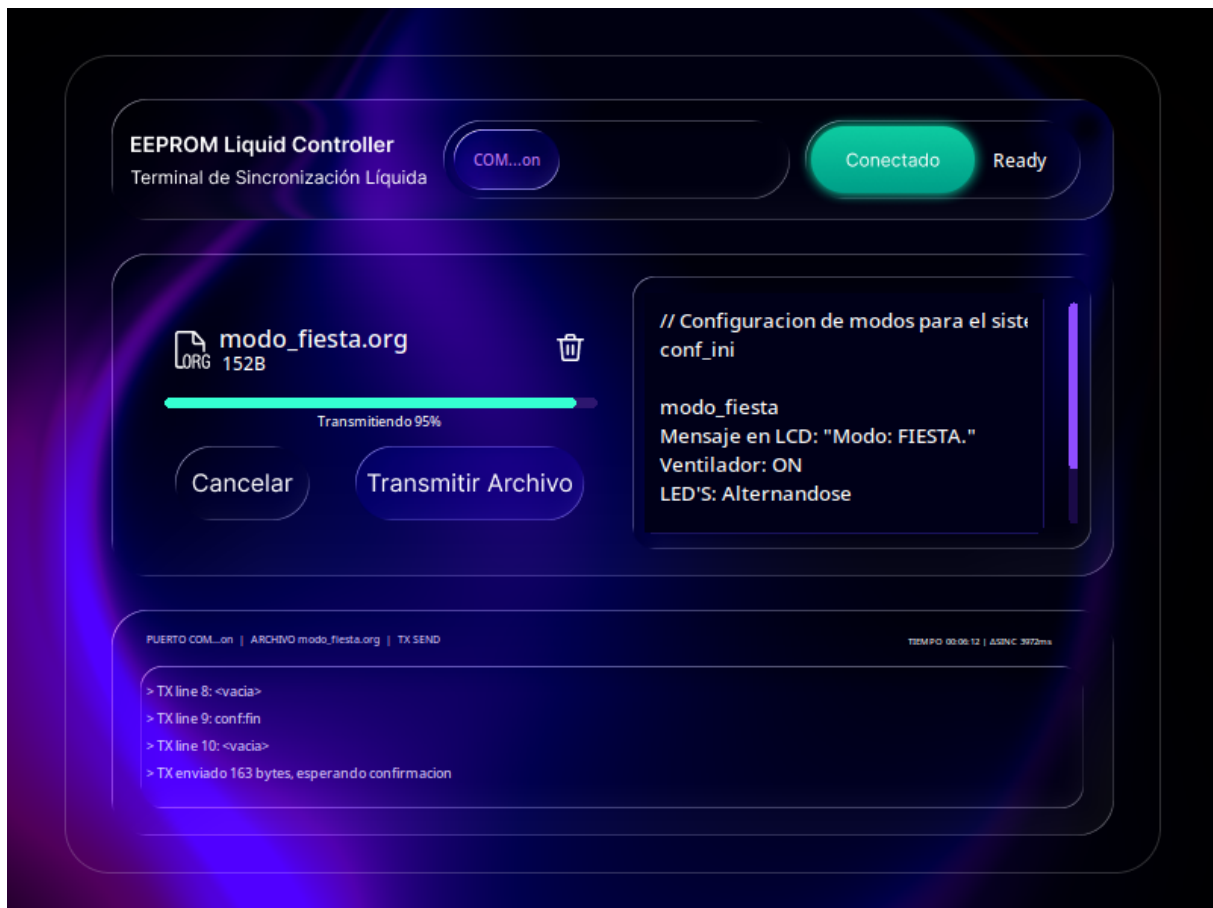


Figure 22: Archivo cargado en la aplicación de escritorio.

La tercera captura corresponde a la fase de transferencia. En ese punto el Arduino ya dejó de interpretar entradas como comandos normales y las procesa como líneas de configuración.

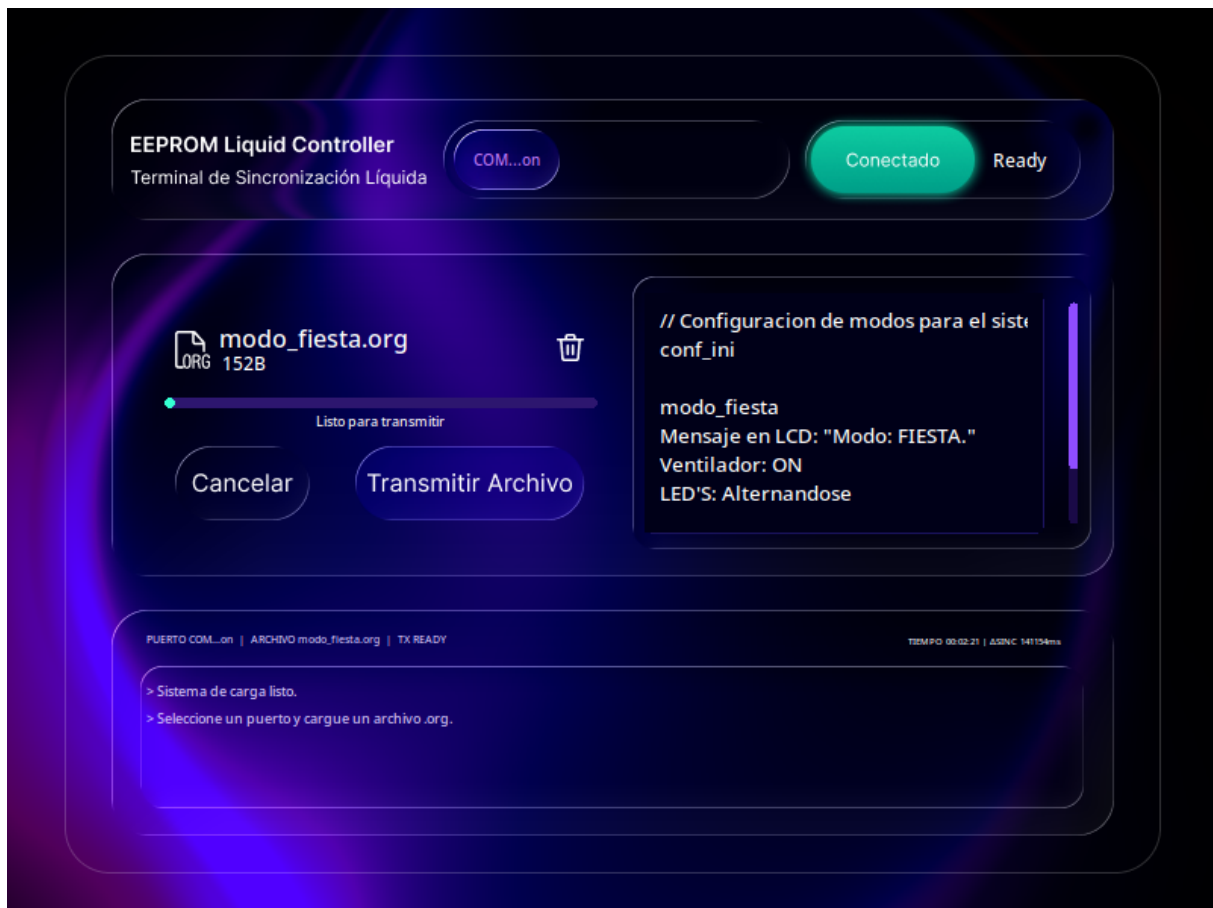


Figure 23: Transferencia activa del archivo .org.

La aplicación móvil se muestra en dos pasos porque el flujo real tiene dos fases: primero seleccionar el dispositivo Bluetooth correcto y luego enviar acciones concretas. El sketch solo necesita recibir cadenas como `modo_fiesta`, `modo_noche` o `estado`.

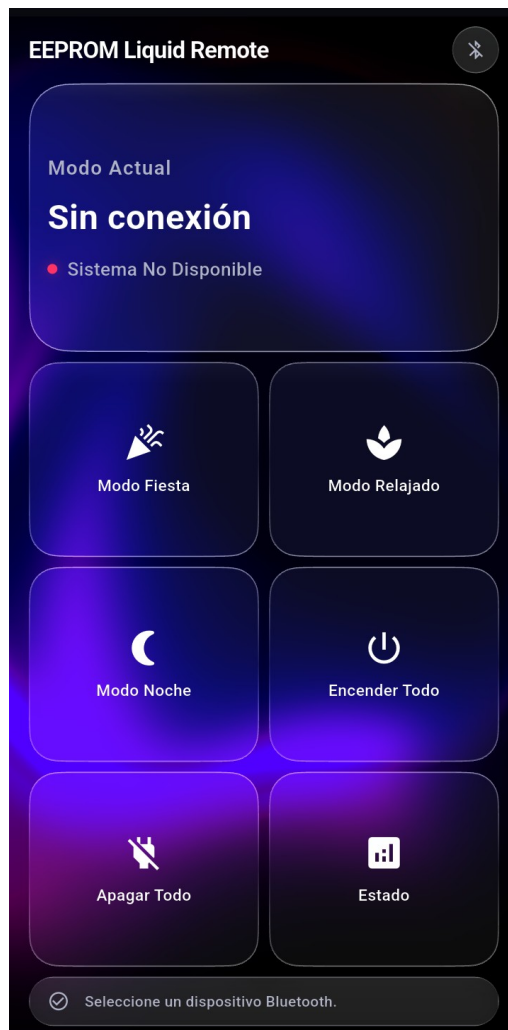


Figure 24: Panel principal de la aplicación móvil.

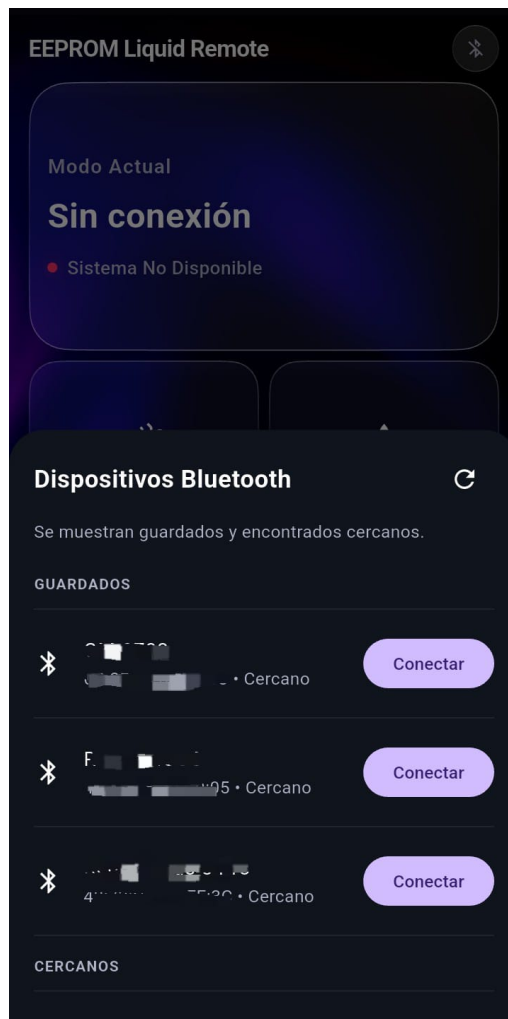


Figure 25: Pantalla de dispositivos Bluetooth de la aplicación móvil.

9 Fragmentos clave del código Arduino

Esta sección reúne los bloques del sketch que gobiernan el comportamiento general del sistema.

9.1 Aplicación de escenas

La rutina `applyMode()` enlaza EEPROM, LEDs de estado, LCD, ventilador e iluminación. Primero valida que la escena leída tenga firma correcta, luego actualiza el estado general del sistema y finalmente activa las salidas físicas.

```

void applyMode(byte mode) {
    SceneConfig scene = readSceneFromEeprom(mode);
    if (scene.magic != EEPROM_MAGIC) {
        systemError = true;
        digitalWrite(led_rojo, HIGH);
        showLcd("Modo sin config", modeName(mode));
        return;
    }

    activeMode = mode;
    systemError = false;
    digitalWrite(led_rojo, LOW);
    digitalWrite(led_azul, HIGH);
    digitalWrite(motor_ventilador, scene.fan == FAN_ON ? HIGH : LOW);
    applyLedPattern(scene.ledPattern);
    showLcd(scene.message,
            scene.fan == FAN_ON ? "Ventilador ON" : "Ventilador OFF");
}

```

Figure 26: Rutina principal de aplicación de una escena almacenada.

9.2 Escenas por defecto

El proyecto no depende exclusivamente de una carga manual inicial. Si la EEPROM todavía no contiene una configuración válida, el Arduino crea escenas base para que la maqueta pueda arrancar operativa.

```

void seedDefaultScenesIfNeeded() {
    SceneConfig first = readSceneFromEeprom(MODE_APAGAR_TODO);
    if (first.magic == EEPROM_MAGIC) return;

    writeDefaultScene(MODE_FIESTA, FAN_ON, LEDS_ALTERNATING, "Modo: FIESTA.");
    writeDefaultScene(MODE_RELAJADO, FAN_OFF, LEDS_ON, "Modo: RELAJADO.");
    writeDefaultScene(MODE_NOCHE, FAN_OFF, LEDS_OFF, "Modo: NOCHE.");
    writeDefaultScene(MODE_ENCENDER_TODO, FAN_ON, LEDS_ON, "Modo: TODO ON.");
    writeDefaultScene(MODE_APAGAR_TODO, FAN_OFF, LEDS_OFF, "Modo: TODO OFF.");
}

```

Figure 27: Inicialización automática de escenas por defecto.

9.3 Comandos operativos

Además de aplicar modos, el sketch soporta comandos puntuales que no requieren recargar la configuración completa, por ejemplo consultar estado o controlar la puerta de forma directa.


```

if (command == "estado") {
    printStatus();
    return;
}
if (command == "abrir_puerta") {
    openDoor();
    return;
}
if (command == "cerrar_puerta") {
    closeDoor();
    return;
}

```

Figure 28: Fragmento del manejo de comandos operativos del sistema.

10 Recursos y herramientas utilizadas

10.1 Hardware

- Arduino Uno o compatible
- Módulo Bluetooth HC-06
- Pantalla LCD I2C 16x2
- LEDs para ambientes
- LEDs azul, verde y rojo de estado
- Motor DC para ventilador
- Servomotor para puerta
- Resistencias y cableado
- Fuente de alimentación para prototipo

10.2 Software

- Arduino IDE o `arduino-cli`
- Proteus
- Python 3.11 para la app de escritorio
- Flutter para la app móvil
- Git y GitHub para control de versiones

11 Cronograma de trabajo

Table 6: Fechas base del proyecto según el PDF.

Tarea	Fecha
Asignación del proyecto / entrega del enunciado	18/04/2026
Fecha de entrega	02/05/2026

12 Presupuesto

El PDF exige presupuesto estimado y fotos de respaldo. En este repositorio no se encontró factura ni tabla de costos final, por lo que se deja la estructura lista para completar con los precios reales del equipo.

Table 7: Plantilla de presupuesto estimado.

Componente	Cantidad	Costo unitario	Subtotal
Arduino Uno o compatible	Pendiente	Pendiente	Pendiente
HC-06	Pendiente	Pendiente	Pendiente
LCD I2C 16x2	Pendiente	Pendiente	Pendiente
Servomotor	Pendiente	Pendiente	Pendiente
Motor DC	Pendiente	Pendiente	Pendiente
LEDs y resistencias	Pendiente	Pendiente	Pendiente
Material de maqueta	Pendiente	Pendiente	Pendiente
Cableado y fuente	Pendiente	Pendiente	Pendiente
Total			Pendiente

13 Resultados y observaciones

- El sketch ya define almacenamiento persistente para cinco modos con direccionamiento fijo.
- La app de escritorio ya cubre selección de puerto, vista previa y envío serial.
- La app móvil ya contempla conexión Bluetooth, envío de comandos y retroalimentación visual.
- El uso de `millis()` para el patrón alternado evita bloquear el ciclo principal del Arduino.
- Existe una discrepancia entre el PDF y el sketch actual: el PDF describe `modo_relajado` con LEDs apagados, pero el código lo inicializa con LEDs encendidos.
- En la app de escritorio existe un parser `.org` reservado, pero aún está marcado como `NotImplementedError`; la validación real la hace actualmente el Arduino.

14 Conclusiones

- La solución cumple con la arquitectura general pedida en el proyecto: configuración externa, persistencia en EEPROM, control remoto y retroalimentación local.
- La separación entre Arduino, app de escritorio y app móvil facilita mantenimiento, pruebas y demostración del flujo completo.
- El mapa fijo de EEPROM y el formato `.org` documentado reducen ambigüedades para futuras pruebas o ampliaciones.
- Para cerrar completamente la entrega solo faltaría consolidar la evidencia física final de la maqueta y completar el presupuesto real.

15 Anexos

15.1 Archivos de apoyo

- PDF del enunciado del proyecto.
- Sketch Arduino principal.
- Proyecto Proteus y export del esquema.
- Ejemplo de archivo `.org`.
- Capturas de la app de escritorio y de la app móvil.

15.2 Pendientes para cierre final

- Agregar fotos de la maqueta física terminada y del equipo durante el desarrollo.
- Completar la tabla de presupuesto con costos reales o factura.
- Si se requiere entrega final en PDF, compilar este archivo y revisar saltos de página y tamaño de imágenes.